

Peakflo — Expense Account Classification

ML Engineer Intern Take-Home: Model, Evaluation, Findings and Improvement Plan

Executive Summary

The objective of this work is to automate the classification of expense transactions into the correct accountName, reducing the amount of manual categorization required by the finance team while maintaining a reliable and auditable evaluation process. The supplied dataset contains 4,894 expense transactions, 337 vendors, 103 account-name classes, and 97 account IDs. The final leakage-safe LinearSVC solution achieved **85.28% accuracy on the untouched test set**, exceeding Peakflo's stated 85% minimum requirement, with **85.63% group-aware cross-validation accuracy**. Weighted F1 was 84.23%, macro F1 was 71.64%, and balanced accuracy was 74.13%. The evaluation also exposed a meaningful generalization limitation: training accuracy was 98.78%, compared with 85.28% on the test set, a 13.50 percentage-point gap. This indicates that further improvement should focus on generalization rather than simply increasing model complexity. I therefore view the current model as a strong, reproducible baseline that clears the assignment threshold, while leaving a clear path toward a more robust system capable of targeting 90%+ performance through better text representations, vendor-independent features, confidence-aware routing, hierarchical classification, temporal validation, and targeted treatment of the most difficult account categories.

1. Data Analysis

1.1 Dataset and business problem

Peakflo processes expense transactions that must be assigned to the appropriate accounting account. In the supplied data, the prediction target is accountName. In a production workflow, an automated classifier could receive a newly created expense transaction and propose the appropriate account before the transaction reaches a finance reviewer.

The dataset contains:

Property	Observation
Transactions	4,894
Unique vendors	337
Unique account names	103
Unique account IDs	97
Unique transaction `_id` groups	3,181
Exact duplicate rows	10
Rows participating in duplicate groups	18
Missing `itemDescription` values	11
Missing values in other core fields	0

The available predictive information is primarily contained in itemName, itemDescription, vendorId, and itemTotalAmount.

Two fields require special treatment. accountId is not used as a feature because it directly identifies the accounting account and would therefore constitute target leakage. _id is also not supplied to the classifier; it is used only as a grouping key so that related records cannot be split across training and validation/test partitions.

1.2 Data quality observations

The dataset contains 4,894 rows but only 3,181 unique `_id` groups. There are 801 multi-row groups, and 388 groups contain more than one `accountName` label. This makes ordinary random row-level splitting unsafe: related transactions could appear in both training and validation/test data and produce an overly optimistic estimate of generalization.

There are only 11 missing `itemDescription` values, representing approximately 0.22% of the dataset. This is a small amount of missingness and does not justify deleting records. Instead, the modeling pipeline handles missing text safely.

There are 10 exact duplicate rows, involving 18 rows in duplicate groups. Duplicate analysis is useful because repeated transactions can otherwise make a model appear stronger than it is. The evaluation therefore retains the original records while controlling grouping during validation.

1.3 Class imbalance

The target contains 103 account categories with a strongly imbalanced distribution.

The largest class is:

611202 Online Subscription/Tool — 1,179 transactions

Other large classes include:

- 132098 IC Clearing account — 706
- 619203 Supplies/Expenses — 225
- 134001 Prepaid Operating Expense — 193
- 614123 Employee On Record — 175

At the other end of the distribution:

- 16 classes have fewer than 2 observations.
- 34 classes have fewer than 5 observations.
- 43 classes have fewer than 10 observations.
- 59 classes have fewer than 20 observations.

This is an important modeling constraint. A conventional accuracy score can be dominated by the largest categories, while extremely rare categories may have insufficient examples to learn reliable decision boundaries. For this reason, accuracy is reported together with weighted F1, macro F1, balanced accuracy, and category-level error analysis.

1.4 Vendor signal

There are 337 unique vendors, and the vendor analysis shows that vendors can be highly informative but are not uniformly associated with a single account. Some vendors have high account purity while others appear across multiple accounting categories.

This observation supports including `vendorId` as a feature, but it also creates a generalization risk. A production model should benefit from vendor information when the vendor is known without becoming dependent on vendor memorization when a new supplier appears.

The final diagnostic confirms this issue. On the frozen test set, 976 transactions belonged to vendors observed during development and achieved 87.70% accuracy.## 1.5 Transaction-text characteristics

The transaction fields contain substantial textual information. The analysis measured character length, token counts, and digit counts for both `itemName` and `itemDescription`. Median transaction names and descriptions contain approximately five tokens, while some records contain substantially longer descriptions.

Digits are also common in transaction text. This is useful because accounting descriptions frequently contain dates, plan identifiers, branch identifiers, product numbers, or other structured fragments. Character-level text representations can therefore capture patterns that word-level representations may miss.

2. Methodology

2.1 Leakage prevention

The modeling process follows three strict rules.

Rule 1 — Exclude `accountId`

`accountId` is an alternative representation of the target account. Supplying it to the model would make the classification task artificially easy and would not represent a legitimate prediction scenario.

Rule 2 — Use `_id` only for grouping

`_id` is used to prevent related observations from crossing evaluation boundaries. It is never supplied as a predictive feature.

Rule 3 — Fit target-derived information only on training data

Vendor-to-account and text-to-account mappings are created only from the relevant training partition. Test labels are never used to construct a feature or rule.

These controls are essential because the goal is not merely to produce a high numerical score; it is to estimate how the system can perform on genuinely unseen transactions.

2.2 Validation strategy

The dataset is first divided into development and final test data. The final test set is frozen before model selection.

Within the development partition, group-aware cross-validation is used. The folds have zero group overlap, meaning an `_id` group cannot occur in both the training and validation side of a fold.

This approach produces a more conservative and realistic estimate than ordinary random row-level cross-validation.

The selected LinearSVC configuration was evaluated using three group-aware folds during the faster hyperparameter search. The selected configuration was `C=1.0` with no class weighting.

2.3 Feature engineering

The model combines several sources of information:

Text features

`itemName` and `itemDescription` are normalized and combined into a transaction text representation.

Two TF-IDF representations are used:

- Word-level TF-IDF to capture meaningful words and short phrases.
- Character-level TF-IDF to capture spelling variations, abbreviations, identifiers, and partial word patterns.

This is particularly suitable for expense data because descriptions often contain semi-structured strings rather than clean natural language.

Vendor features

`vendorId` is one-hot encoded with unknown vendors handled safely. This allows the model to learn vendor-specific accounting tendencies without failing when a new vendor appears.

Numerical features

The transaction amount is transformed using a signed/log-style representation to reduce the influence of extreme amounts. Additional features capture text length, token count, and digit count for the name and description fields.

These features provide lightweight structural information alongside the high-dimensional text representation.

2.4 Algorithm choice

LinearSVC was selected as the main model because the dataset is small enough that a high-dimensional sparse text representation is practical, while the classification problem contains many classes and substantial textual signal.

LinearSVC is also computationally efficient compared with many nonlinear alternatives when operating on sparse TF-IDF matrices. The approach provides a strong baseline without requiring a large amount of training data.

A CatBoost alternative was investigated conceptually and implemented during experimentation, but the native text-processing configuration was computationally expensive on the available environment. Rather than force an inefficient model into the final submission, the final reported result uses the validated LinearSVC approach.

2.5 Handling imbalance and rare classes

Rare account categories were retained rather than removed. Removing them would make the benchmark easier but would change the original business problem.

The evaluation therefore reports macro F1 and balanced accuracy in addition to overall accuracy. This exposes whether a model is performing only on the largest accounts or is also learning smaller categories.

The analysis also identifies the categories requiring additional data or specialized features in future iterations.

3. Results

3.1 Final performance

The final selected model was **LinearSVC**.

Metric	Result
Training accuracy	**98.78%**
Group-aware CV accuracy	**85.63%**
Final test accuracy	**85.28%**
Weighted precision	**84.95%**
Weighted recall	**85.28%**
Weighted F1	**84.23%**
Macro F1	**71.64%**
Balanced accuracy	**74.13%**
Train-test gap	**13.50 percentage points**

The final test accuracy of **85.28%** passes **Peakflo's stated 85% assignment threshold**.

The group-aware CV accuracy of 85.63% and final test accuracy of 85.28% are close, differing by only 0.35 percentage points. This consistency is encouraging because it suggests that the evaluation methodology provides a reasonably stable estimate of generalization.

However, the 98.78% training accuracy compared with 85.28% test accuracy reveals a substantial generalization gap. Therefore, the next improvement should focus on reducing memorization and improving robustness rather than simply increasing model capacity.

3.2 Category-level interpretation

Macro F1 of 71.64% is substantially lower than overall accuracy. This is consistent with the strong class imbalance in the dataset: the model performs better on well-represented accounts than on extremely rare categories.

This distinction is important for deployment. An automated system should not treat an 85% aggregate accuracy result as evidence that every account category is equally reliable.

The next iteration should therefore prioritize the bottom-performing categories and the most common confusion pairs. For high-volume accounts, even a small improvement can produce a meaningful reduction in manual finance review volume. For rare accounts, additional labeled examples may be more valuable than increasing model complexity.

3.3 Generalization by vendor availability

The vendor generalization diagnostic provides another important result:

Vendor status	Test rows	Accuracy
Seen during development	976	**87.70%**

00%** |

This result suggests that vendor information is highly useful but that the current model has limited robustness when the supplier is completely new.

Nevertheless, it identifies a clear engineering opportunity: improve the model's ability to classify expenses from transaction semantics and amount information when vendor history is unavailable.

4. Discussion

4.1 How the model helps Peakflo with real data

The practical value of the model is not limited to producing a prediction. It can become an automated decision-support layer in the expense processing pipeline.

For a newly received expense, the system can:

1. Read the vendor, expense name, description, and amount.
2. Predict the most likely accounting account.
3. Attach a confidence score to the prediction.
4. Automatically process high-confidence predictions.
5. Route low-confidence or ambiguous predictions to a finance reviewer.
6. Store reviewer corrections as new labeled data for future model improvement.

This creates a human-in-the-loop workflow rather than requiring the model to make every accounting decision blindly.

At the current 85.28% test accuracy, approximately 85 out of every 100 transactions in the evaluation set are classified correctly. In a production setting, this should be combined with confidence thresholds and review policies rather than interpreting accuracy as an unconditional automation rate.

The business benefit is that repetitive, high-confidence classifications can be automated while finance specialists spend their time on ambiguous, unusual, or genuinely new transactions.

4.2 Strengths

The main strengths of the approach are:

- The model exceeds the assignment's 85% accuracy requirement.
- The test set remains untouched during model selection.
- Group-aware validation reduces optimistic estimates caused by related records crossing folds.
- `accountId` is excluded to prevent direct target leakage.

- `_id` is used for grouping rather than prediction.
- Rare categories are retained.
- Multiple metrics are reported rather than relying on accuracy alone.
- The feature representation combines textual, categorical, and numerical information.
- The methodology is reproducible and suitable for further iteration.

4.3 Limitations

The largest limitation is the 13.50 percentage-point train-test gap. The model is clearly capable of fitting the development data more closely than it can generalize.

A key limitation is generalization to transaction patterns that differ from those represented in development data. Future evaluation should therefore include stronger vendor-independent features and a larger out-of-distribution validation sample.

The class distribution is highly skewed, and some categories contain fewer than five examples. No model can reliably learn a stable decision boundary for a category with almost no examples without additional information.

The current data also does not provide a clearly usable transaction date for a robust temporal validation experiment. Before production deployment, it would be valuable to evaluate whether a model trained on historical periods continues to perform on future accounting periods.

4.4 Path toward 90%+ accuracy

The current result should be treated as a strong baseline rather than the endpoint. Several improvements could reasonably be investigated.

1. Better text representation

The next iteration should explore stronger representations of `itemName` and `itemDescription`, including richer n-gram configurations, domain-specific normalization, and potentially compact pretrained language-model embeddings.

The objective would be to capture semantic similarity between descriptions that use different wording but refer to the same accounting concept.

2. Vendor-independent model

A dedicated text-and-amount model should be evaluated without `vendorId`. Its purpose would not necessarily be to replace the main model, but to provide a strong fallback when a vendor is unseen.

3. Confidence-aware routing

Instead of using one prediction mechanism for every transaction, the production system could route predictions according to confidence:

- High-confidence, well-supported vendor/text combinations → automated classification.
- Moderate-confidence predictions → secondary model or rule check.
- Low-confidence predictions → human review.

This can improve the precision of automated decisions without forcing the model to guess on ambiguous cases.

4. Hierarchical classification

A hierarchical classifier could first identify a broad accounting family and then classify within that family. This can be useful when several of the 103 categories are semantically related.

Instead of asking one classifier to distinguish all 103 classes equally, the system can reduce the candidate space progressively.

5. Targeted modeling of confusion pairs

The confusion matrix can identify account pairs that are repeatedly confused. Instead of changing the entire model, specialized features can then be created for those specific pairs.

For example, if two subscription-related accounts are frequently confused, the model can use additional text phrases, amount patterns, or vendor/account history specifically relevant to that distinction.

6. More labeled examples for rare accounts

Some categories have extremely small support. Collecting additional examples for these categories may produce a larger improvement than increasing model complexity.

Active learning can help: send low-confidence transactions to finance reviewers and prioritize the corrected examples that provide the most information.

7. Temporal validation and drift monitoring

A production expense classifier should be evaluated across time. Vendors, subscription plans, accounting policies, and expense descriptions can change.

A future system should monitor:

- prediction confidence
- new-vendor rate
- class distribution
- text distribution
- correction rate from finance reviewers
- accuracy by accounting period

This would make the classifier maintainable rather than treating the take-home model as a one-time artifact.

5. Deployment and Business Considerations

A production architecture could place the classifier after transaction ingestion and before final accounting approval.

The recommended workflow is:

Transaction ingestion → feature extraction → account prediction → confidence/risk check → automatic posting or finance review → feedback collection → periodic retraining

The model should not be treated as an unconditional authority. Accounting classification can have financial and compliance consequences, so low-confidence cases should remain reviewable.

A particularly useful production metric would be **automation coverage at a target precision** rather than accuracy alone. For example, Peakflo could determine the confidence threshold at which automated classifications meet an internal quality requirement, then send the remainder to finance reviewers.

This creates a measurable trade-off between automation and review workload.

The feedback loop is also valuable. Every corrected prediction becomes a potential labeled training example. Over time, this can improve performance on new vendors, new expense patterns, and previously rare account categories.

6. Conclusion

The developed solution achieves **85.28% accuracy on the untouched test set**, exceeding the assignment's required 85% threshold, while using a leakage-safe group-aware evaluation methodology. The close relationship between group-aware CV accuracy (85.63%) and final test accuracy (85.28%) provides confidence that the reported performance is not simply an artifact of an optimistic split. At the same time, the 13.I would therefore treat this model as a strong baseline for Peakflo rather than the final ceiling. The most promising next steps are stronger

vendor-independent text modeling, confidence-aware routing, hierarchical classification, targeted treatment of recurring confusion pairs, additional labeled data for rare categories, and temporal/drift-aware evaluation. These improvements are aimed not only at pursuing a 90%+ benchmark, but at building a classifier that can provide reliable automation on the changing, real-world expense data encountered by a finance platform.